What a trained 1024 × 16 net looks like next to a random one, through the cumulant chain (submission #331937)

James Henry (jamesrahenry), with Claude Code

ARC White-Box Estimation Challenge 2026, Phase 2 algorithmic-contribution write-up. 2026-10-28

Submission #331937 (jamesrahenry): adjusted 7.04e-9, final-layer MSE 2.18e-8 at 32 % of budget, 0/100 failures, regraded bit-identical under flopscope 0.12.3 as #337164. A clean-room port of ARC's factorized K=3 chain, built from the paper and the reference code without reading any participant's estimator; it reached the V18 band and nothing in it is new. Release, defect log and the levers we tried and closed: [LINK: estimator release]. This post is about what we did with it afterwards.

The idea

Every estimator on this board is calibrated on He-Gaussian weights, and that is the challenge as posed. But the chain is also a very precise null model: it says what a 1024 × 16 ReLU net looks like with zero learned structure. So we trained a few hundred nets at exactly that architecture (Gaussian-mixture and MNIST-class tasks, AdamW, weight decay and training length varied, every init kept), froze #331937, and pointed it at them. Corpus, code, preregistrations and result files are public: https://github.com/jamesrahenry/Model_Zoo_Cartography. Two pictures carry what we found. Both compare a fresh He init with the same architecture trained for 20 000 steps, once without weight decay and once with 0.3.

1. Training builds a relay through the net, and you can see it with no reference at all

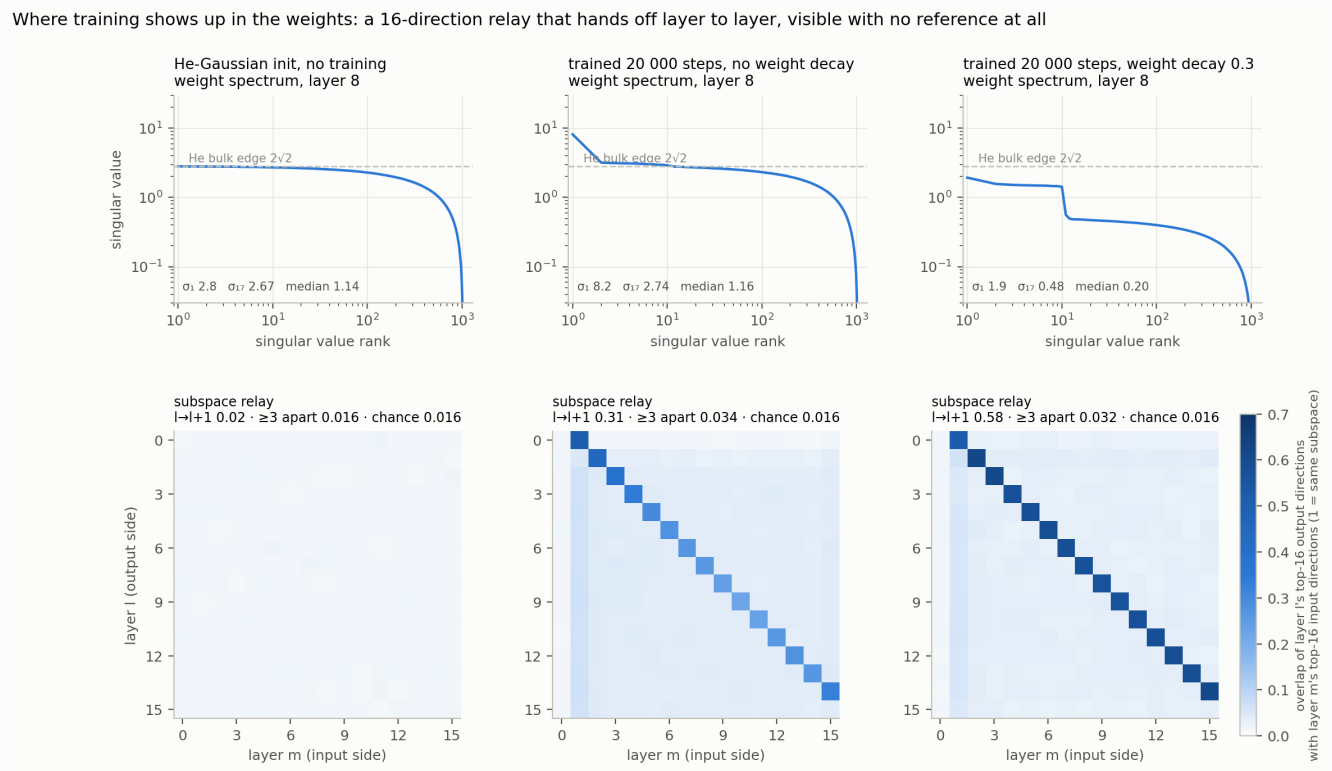


Figure 1: Top: singular values of the layer-8 weight matrix. Bottom: for every pair of layers, how much layer l's top-16 output directions overlap layer m's top-16 input directions (squared cosine, 1 = same subspace, chance 0.016).

In the random net the overlap matrix is flat at chance. In the trained nets there is one bright superdiagonal and nothing else: each layer hands a 16-dimensional subspace to the next layer (overlap 0.31 undecayed, 0.58 decayed) and to no layer further away (0.03 three layers on). Training threads a sixteen-link relay through the net, and the channel turns at every link rather than persisting as one shared subspace. Across 51 corpus nets, 92 % show this chaining at ten times chance from the final weights alone and none of their own inits do (preregistered, I2-P1); it is there in undecayed nets too, where the spectrum shows almost nothing. The spectrum explains the visibility. The random layer is a Marchenko-Pastur bulk ending at $2\sqrt{2}$. Without decay one spike rises above the bulk and the rest of the update hides inside it. With decay the bulk shrinks to 0.18 of its size and ten spikes stand clear. The shrink factor is exactly AdamW's $\exp(-\mathrm{lr}\cdot\mathrm{wd}\cdot t)$ on 108 of 108 nets with decay (preregistered, Q1), and the median singular value of a layer divided by a He matrix's recovers it to 0.004 with no init in hand, except in the first layer, which training rewrites.

What the weights alone do not give is the decomposition. We tried to cut the relay out of the final weights with no reference, per layer and as one shared subspace, and could not: keeping the recovered directions and re-randomising the rest preserves the function in 3 of 64 draw-dependent nets. With the net's own init as the reference it works: about sixteen directions per layer, the part of the random remnant that overlaps the trained update, restores the function, and a random subspace of the same size needs 4 to 32× more. Judged by the chain's own metric instead of accuracy, though, nothing is discardable in low-decay nets: matching the chain's error needs essentially all 1 024 directions in three quarters of them. The weights say *that* training left structure and roughly where. Which directions, and whether the rest is inert, needs a reference and a forward pass.

2. Training switches most of the layer off, and the random-net chain fails exactly on what is left

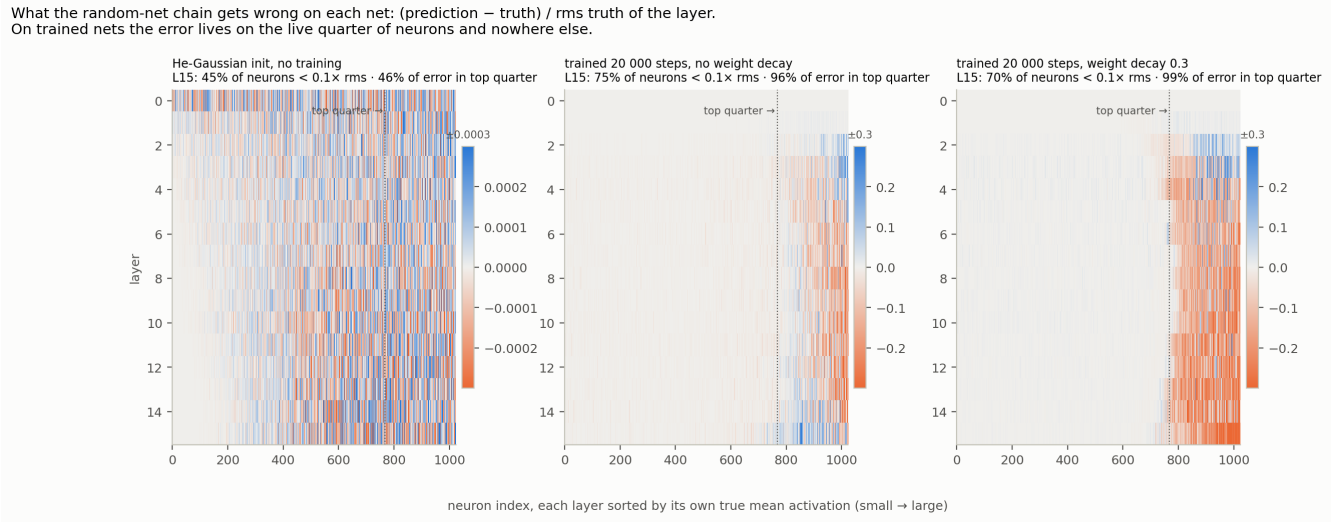


Figure 2: The frozen chain's error on each net, (prediction − truth) / rms truth of the layer, one row per layer, each row's neurons sorted by their own true mean activation. Dotted line at the top quarter. Note the scale: ±3e-4 for the init, ±0.3 for the trained nets.

On the init the chain does what it was built for: error at the 1e-4 level, spread evenly across all neurons. On the trained nets three quarters of each layer is effectively off (70 to 75 % of layer-15 neurons sit below a tenth of the rms) and the whole output lives on the remaining quarter. The chain's error lives there too, 96 to 99 % of it, with a sharp boundary, and it is a thousand times larger. On the decayed net it is same-signed, the prediction about 12 % short of the truth on every live neuron, so a single per-layer scalar fitted on *other* tasks removes most of it (held-out coherent share 0.88 to 0.92; cross-family transfer cuts the last-layer MSE to 0.07 to 0.27×). On the undecayed net it is mixed-sign and no scalar helps. Across the full grid the coherence tracks how much of the init decay has removed (Spearman 0.85 against $-\ln \alpha$) and not how far the task has moved the weights (0.37 against the update size), with one regime we cannot yet explain: short undecayed training is coherent too. The direction the error takes, along the predicted mean, is the one megan_park's survival law (18282, confirmed by dogus_ozel 18281 and AndreasHad04 18292) says the chain cannot forget; on random weights it carries about 1 % of the error, on trained weights nearly all of it.

What we take from it, for now

A trained net at this architecture is not a perturbed random net. It is a sparse live set and a sixteen-direction relay riding on a random bulk that still holds most of the weight energy, and the chain, derived for the bulk, is right about the bulk and wrong about the relay. Two things follow that we would want to check before believing further. By the same closed form, standard LLM pretraining (weight decay ~0.1, learning rate ~3e-4, well over 1e5 steps) gives $\alpha \approx 0$, so the regime where the function depends on its particular random draw may not exist in large models at all, and the right null for one of those is its own early checkpoint rather than a fresh init. And the live-quarter picture was measured on 17 nets tonight and is not preregistered; the rest above is. One chain, one input law (N(0, I)), mostly Gaussian-mixture tasks, and the chain is fragile off its distribution (non-finite on every 2-class net, on depth 24 inside the budget, and at lr 1e-3; 0 of 14 inits failed). The corpus is public so that someone can run a different estimator on it.

Credits and tools

ARC (Wu, Lecomte, Winer, Robinson, Hilton, Christiano) for the framework and reference code; 504aldo (18218) for the public line our port was checked against; megan_park (18282), dogus_ozel (18281) and AndreasHad04 (18292) for the survival law; Artyom_Mazur (18249) and il.miscusi (18248) for Proposition 1 and the scale mode; michael_anderson7 (18272) for crediting the Phase 1 cancellation account; keenanpepper for the public corpus; AIcrowd and mohanty for the rulings. Claude Code (Anthropic; Claude Opus and Fable 5) wrote and ran the code and drafted this text under James Henry's direction; James set the direction, chose the pivot to trained weights, rejected the accuracy-only extraction metric, and reviewed every claim. Preregistration files were written by the model and committed before the runs; hashes are in the repository.